DHI Desk Design v1: Tutor, Library, Reuse Engine

Oct 13, 2026 · @Dhi

DHI Desk is a syllabus-grounded tutor for classes 8 to 12 that knows each student's progress, and it is built so that anything generated once is checked, stored and reused for every later student who needs it. The language models are swappable engines inside it; DHI owns the content, the student model and the loop.

What the desk is

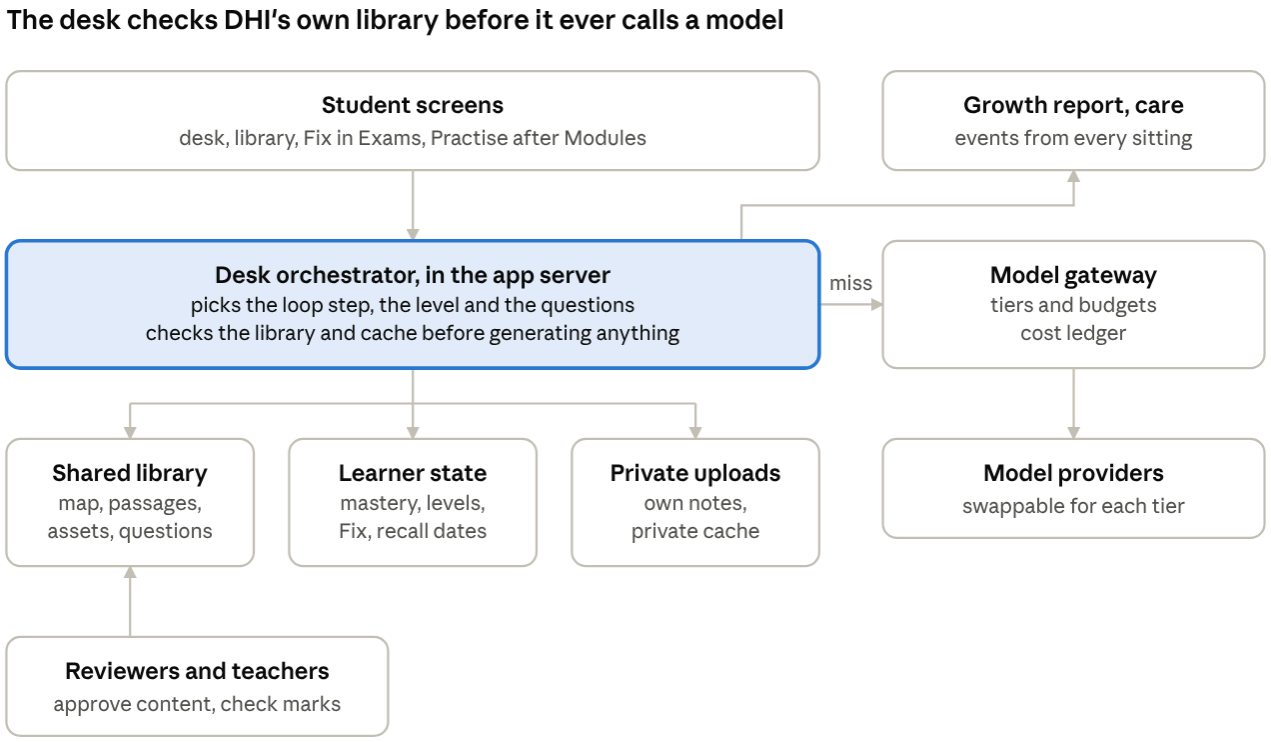
A general chatbot answers a question. The DHI desk teaches a topic until the student can do it, remembers what they got wrong, and brings it back at the right time. Its advantage over Claude, Grok or any other model comes from three things DHI owns and they don't: a mapped syllabus, a model of each student, and a growing library of reviewed content.

Rules the desk keeps

1. **Syllabus first.** Answers come from DHI's library and the student's own uploads before the model's memory. Every explanation cites the chapter it came from.
2. **Generate once, check, reuse.** Anything not personal (an explanation, a story, a diagram, a question) is stored after it is made and served to the next student who needs it. A second request for the same thing costs no model call and no coins.
3. **Shared content never contains a student.** Personal touches (their name, their past mistake) are added on top of a shared asset at the last moment and never stored in the shared library.
4. **Code marks MCQs; the model never does.** Answer keys live on the server.
5. **Hints, not answers, on assigned work.** The exam lock still closes the desk while a paper is open.
6. **Feelings are not handled by the model.** Distress written into the desk goes to the care rules and a counselor.
7. **The model is a replaceable part.** Every call goes through one gateway, so the provider can change without touching the desk.
8. **Every sitting feeds the report.** Subject, topic, mode and right or wrong are recorded as events (metrics A10 to A14 in the master spec).

System map

The desk lives inside the dashboard's app server and database; only the model providers are outside.



desk system map · orchestrator, library, learner state, gateway

A request goes to the orchestrator, which reads the library, the student's learner state and their uploads. It calls the gateway only on a miss, and writes events that feed the Growth report and the care rules. Reviewers turn generated content into trusted shared content.

Knowledge graph and library

The foundation is a map of the syllabus for classes 8 to 12 across all subjects, with every piece of content pinned to a place on it. This is what lets the desk search any chapter, teach at the right level and reuse what it has made.

The map

- Board → class → subject → chapter → topic → concept. A topic is the unit everything hangs on: questions, explanations, mastery, the heat map.
- **Prerequisite links** between topics ("moments" before "torque"), so the desk can step back when a student is missing a foundation.

- **Misconceptions** per topic ("the stomata cook the sugar"), each with a short correction. Wrong MCQ options are tagged with the misconception they test.
- **Cross-board links**, so the same idea in CBSE and a state board shares content where the syllabus matches.

What the library holds per topic

Asset	What it is	Made by
Source text	the chapter text, split into short passages with page references	ingestion from books DHI has rights to
Explanation	at 3 levels (foundation, standard, stretch), in English and Hinglish	generated, then reviewed
Story	the idea told as a short narrative, 4 to 6 panels	generated, then reviewed
Visual	a labelled diagram or flow, stored as a structured spec and drawn by the app	generated as a spec, reviewed
Real-life use	2 or 3 places the idea shows up in daily life, with sources	generated with web sources, reviewed
Questions	MCQs with answer key and tagged wrong options; short and long written questions with key points	generated or imported, reviewed
Worked example	a solved problem, step by step	generated, reviewed

How the library is filled

1. Ingest the source books for each board and class: extract text, split into passages, tag each passage to topics, store with page numbers.
2. Pre-generate the core assets for each topic in batches, starting with the most used chapters.
3. A subject reviewer checks each asset once. Reviewed assets are served to everyone; unreviewed ones only after passing automated checks, and are marked "new".
4. Usage decides what gets generated next: a topic many students open is filled out first.

The library is browsable in the dashboard by class, subject and chapter, so it is also a student-facing study library.

Student uploads

Students can upload their own notes, worksheets or textbook pages, and the desk teaches from them. Uploads stay private to the student, so their content is never put into the shared library.

Flow

1. The student uploads a PDF or photo (size and type checked; scanned pages go through text recognition).
2. The text is split into passages and each passage is tagged to topics on the map.
3. A fingerprint (hash) of the file is stored. If it matches a book already in the library, the desk uses the library's reviewed content instead of generating again.
4. Passages go into the student's private search index.
5. The desk can now answer "from my notes", quiz on them, and link each passage to the right library topic for explanations and practice.

Reuse rule for uploads. Anything generated *from* a private upload is cached only for that student (the cache key includes the file hash). If two students upload the same file, it is reused only when the file matches a library book. The same upload never becomes shared content.

Safety. Upload text is treated as data, never as instructions to the model, so a worksheet that says "ignore your rules" changes nothing.

The learning loop and every desk mode

The modes are steps of one loop on a topic, not a menu of separate tools. A student can start at any step, and the desk always offers the next one.

1. **Understand:** explanation at the student's level.
2. **See it:** a story or a diagram.
3. **Use it:** where it shows up in real life.
4. **Practise:** MCQs and written questions.
5. **Fix:** go back over every wrong answer until it is right.
6. **Remember:** short recall rounds 1, 3, 7 and 21 days later.

Mode	Student asks	What comes back	Reused from the library?	Model used on a miss	Coins
Search	"Ohm's law", "chapter 4 physics"	the topic page: map position, assets available, the student's mastery	always; no model call	none	0

Mode	Student asks	What comes back	Reused from the library?	Model used on a miss	Coins
Understand	"explain torque"	explanation at their level, with chapter citation	yes, by topic, level and language	large	1 on a miss
See it: story	"tell photosynthesis as a story"	4–6 panel story	yes	large	5 on a miss
See it: diagram	"show me the heart"	labelled diagram drawn by the app	yes	large, structured output	8 on a miss
Use it	"where is this used?"	2–3 real-life uses with sources	yes	large with web search	2 on a miss
Practise: MCQ	"test me on moles"	5–10 questions from the bank at the right difficulty	yes, from the question bank	small, only if the bank is short	0 from the bank
Practise: written	"give me a long answer question"	question, then marked answer with missed key points	question yes; marking is per answer	large for marking	2
Fix	after a wrong answer	why it was wrong (from its misconception tag), a simpler example, a similar question	yes	small	0
Remember	due recall items	3–5 quick questions on topics due	yes, from the bank	none	0
Ask (free chat)	any question	grounded answer with citations	semantic cache of past answers on that topic	large	1 on a miss
Plan	"plan my week for the Physics test"	a day-by-day plan built from due topics and weak spots	no, personal	small	1
From my notes	"quiz me on my upload"	questions from the student's own file	private cache only	small	2 on a miss

Coins follow the rule already in the repo: a cache hit costs nothing, and clarifying questions are free.

Adaptivity: learning at each student's level

The desk sets the level from what the student can actually do, measured per topic, never from a label on the student. Three numbers per student per topic drive everything.

1. Mastery (0 to 1). Starts at 0.3. Each answer moves it toward 1 (right) or 0 (wrong), by an amount that depends on the question's difficulty: getting a hard question right moves it more than an easy one. Answers on the desk, daily papers and module checks all count. Internalised = mastery 0.85 or above, held across 2 attempts at least 3 days apart.

2. Level. Mastery under 0.4 → foundation explanations and easier questions; 0.4 to 0.75 → standard; over 0.75 → stretch. The student can always ask for simpler or harder.

3. Next review date. After a right answer the gap grows (1 → 3 → 7 → 21 → 45 days); after a wrong one it resets to 1 day. "Remember" pulls whatever is due.

Picking questions. The desk picks questions the student should get right about 70 to 85% of the time. Below that is discouraging; above it is too easy to teach anything. Each question's difficulty is learned from every student who has answered it, so the bank calibrates itself.

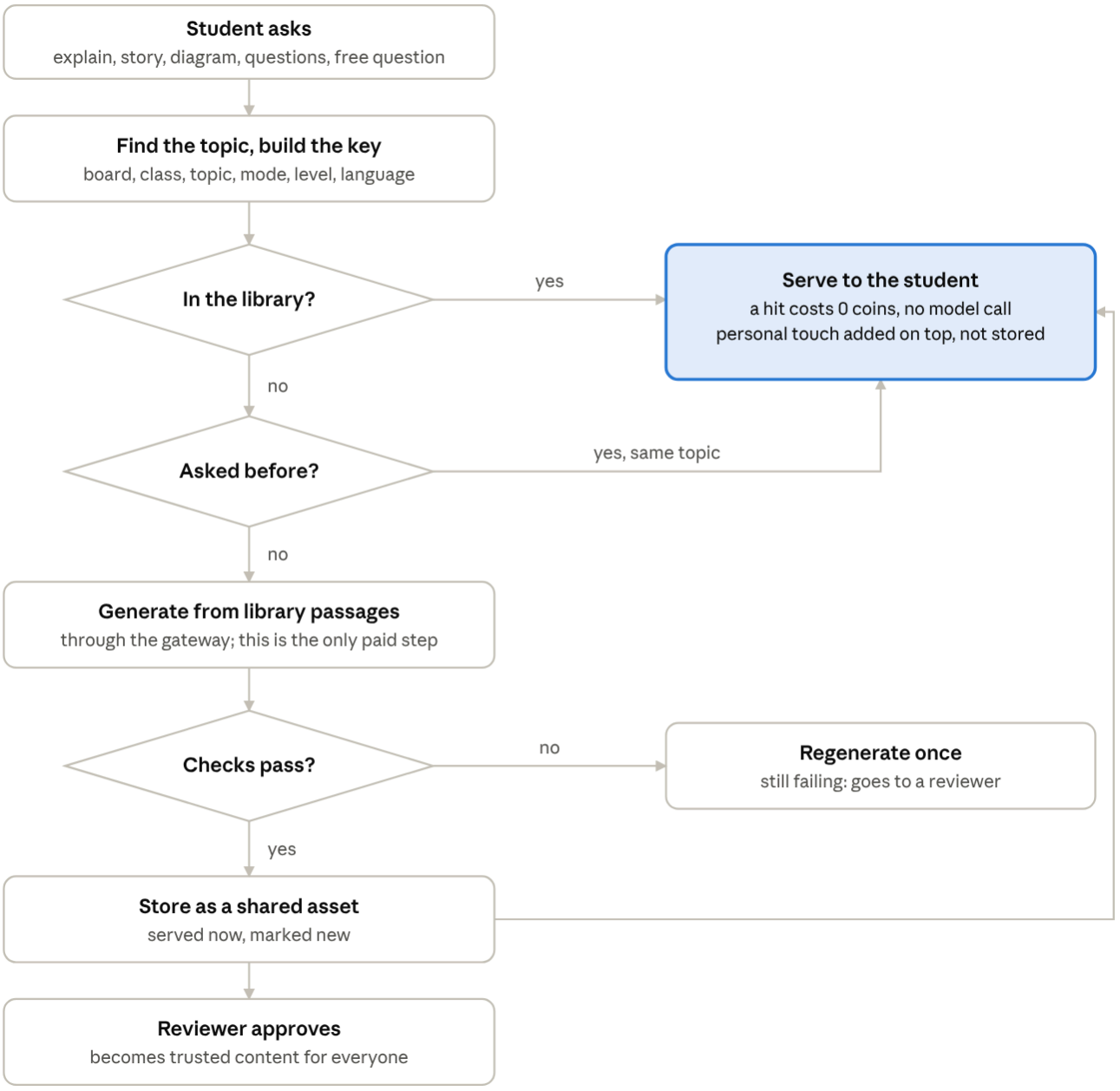
Stepping back. If a student gets 3 questions wrong on a topic and the wrong answers point to a missing prerequisite, the desk offers a 5-minute detour to that earlier topic, then returns.

Learning style, from behaviour. The desk notices which modes are followed by right answers for this student (for example, stories before practice) and offers those first. It also uses their best focus time, from session feedback, when it suggests a plan.

Generate once, reuse forever

The desk pays for a piece of content only the first time anyone needs it. After that every student gets it from storage, free and instantly.

Each piece of content is generated once, checked, then reused



reuse flow · 3 checks before any paid generation

The cache key. board · class · subject · topic · mode · level · language · variant. Two students asking "explain torque, standard level, Hinglish" get the same asset. Variant (1 to 3) lets popular topics hold a few versions, so repeat visitors see something fresh.

"Asked before?" for free questions. A free-text question is turned into a vector and compared with past questions on the same topic. Above a similarity of 0.92, the stored answer is reused. Below it, a new answer is generated and added to the cache. The threshold is tuned by sampling hits each week and checking they really answer the question.

Personal touch. A shared asset never contains a student. Personal lines ("you missed this on Tuesday's paper") come from templates filled at the moment of serving, so they cost nothing and are never stored in shared content.

What is never shared: anything made from a private upload, written-answer marking, and study plans. These go to the student's private cache only.

Versioning. When a reviewer edits or retires an asset, its version increases. Students see the newest version, and nothing old is served after a retirement.

Checking answers

MCQs are marked by code; written answers are marked by the model against key points a teacher agreed, and anything uncertain goes to a teacher.

MCQ. The server compares the choice with the stored answer key. A wrong choice shows its misconception correction straight away, then a similar question. No model call.

Written answers.

1. Each written question stores 3 to 6 key points, each with a weight, plus one model answer.
2. The model receives the question, the key points and the student's answer, and returns a fixed format: for each key point, covered / partly / missing, with the student's words that cover it; plus a confidence score.
3. Marks = Σ weights of covered points (half for partly). The student sees which points they hit and which they missed, with the model answer.
4. Confidence under 0.7, or any mark the student disputes, goes to a teacher's review queue. The teacher's mark replaces the model's and is used to check the marker.
5. Every month, 50 teacher-marked answers per subject are re-marked by the model; if agreement drops below 85% within one mark, the marking prompt is fixed before more marking happens.

Revision after testing. Every wrong or partly-right answer becomes a "Fix" item: the explanation of what was missed, one simpler example, then a fresh question on the same key point. The item stays open until answered right, then joins the spaced recall schedule.

Model gateway and cost control

Every model call goes through one gateway inside the app server, which chooses the model, enforces budgets and logs the cost. The desk never calls a provider directly.

Three tiers

Tier	Used for	Why
Small, fast	mapping a question to a topic, drafting quiz variants, Fix explanations from tags, plans, upload tagging	cheap and quick; most calls
Large	explanations, stories, diagram specs, written-answer marking, free chat	quality matters; and results are mostly reused
Embeddings	search over the library and uploads; finding past answers to similar questions	one-time per passage or question

Providers are set in config per tier, with a fallback for each. Switching provider is a config change, not a code change.

How the cost stays small

- **Reuse first.** Before any generation the gateway checks the asset store and the semantic cache. Most requests for common topics should be hits once the library is filled.
- **Pre-generate core content** in batches, at off-peak prices where providers offer batch discounts, instead of live.
- **Ground with short passages,** not whole chapters, so prompts stay small.
- **Budgets.** A per-student daily cap on model spend, plus the monthly coin allotment already in the repo. When a student's cap is reached, the desk still serves everything already in the library.
- **Cost ledger.** Every call logs model, tokens, cost, cache hit or miss, and which mode asked. A dashboard for you shows cost per student per month and hit rate per mode.

The cost formula to track: monthly model cost per student = (requests that miss the cache × average cost per generation) + marking calls + embeddings. The lever is the hit rate. Set your target price per student, then the ledger shows whether the hit rate is high enough to meet it.

Database tables

The desk adds four groups of tables to the same Postgres database as the rest of the dashboard: the syllabus map, the shared library, the student's own learning state, and the model ledger. Vector search uses the `pgvector` extension, which both Neon and PGLite support, so no separate search service is needed. The six desk tables already sketched in `src/lib/dhi/desk/schema.ts` are kept and extended.

Syllabus map (shared, written by DHI admins)

Table	Key columns
`kg_boards`, `kg_classes`, `kg_subjects`	id, name; class 8 to 12
`kg_chapters`	id, board_id, class, subject_id, number, title
`kg_topics`	id, chapter_id, title, order, summary
`kg_concepts`	id, topic_id, title
`kg_topic_links`	from_topic, to_topic, kind (prerequisite, same_idea_other_board)
`kg_misconceptions`	id, topic_id, wrong_idea, correction

Shared library

Table	Key columns
`library_docs`	id, board, class, subject, title, publisher, rights_note, file_hash, storage_path
`library_passages`	id, doc_id, page, text, topic_ids[], embedding vector
`assets`	id, topic_id, kind (explanation, story, diagram, application, worked_example), level, lang, variant, content jsonb, sources[], status (draft, auto_checked, reviewed, retired), version, model, cost, hits, created_at
`asset_reviews`	asset_id, reviewer_id, verdict, notes, at
`questions`	id, topic_id, kind (mcq, short, long), stem, options[], answer_index (server only), option_misconception_ids[], key_points jsonb, model_answer, difficulty, status, source (generated, imported, teacher)
`question_stats`	question_id, attempts, correct, fitted difficulty, flags
`semantic_cache`	id, topic_id, lang, normalized_text, embedding vector, asset_id, hits

Student learning state (per student, private)

Table	Key columns
`uploads`	id, student_id, file_hash, storage_path, status, library_doc_id (if it matched)
`upload_passages`	id, upload_id, page, text, topic_ids[], embedding vector
`learner_topic_state`	student_id, topic_id, mastery, level, attempts, last_seen, next_review_at, interval_days, misconceptions_seen[]
`desk_attempts`	id, student_id, question_id, source (desk, paper, module), chosen, correct, seconds, hint_used, at
`written_grades`	id, attempt_id, points jsonb, marks, confidence, teacher_mark, status
`fix_items`	id, student_id, question_id, key_point, status (open, fixed), opened_at, fixed_at
`study_plans`, `plan_items`	student_id, week, topic_id, mode, day, done
`private_cache`	student_id, cache_key, asset jsonb (results from uploads; never shared)
`dhi_threads`, `dhi_messages` (existing)	add mode, topic_id, asset_id, cache_hit, gateway_call_id
`dhi_wallets` (existing)	unchanged: allotment, bonus, spent

Model ledger

Table	Key columns
`gateway_calls`	id, student_id (null for pre-generation), mode, tier, provider, model, input_tokens, output_tokens, cost, latency_ms, cache_hit, status, prompt_version
`prompt_templates`	id, mode, version, text, output_schema, active
`budgets`	scope (student, institute, global), period, cap, spent

APIs

All are TanStack Start server functions in `src/server/desk/`, each starting with `requireUserId()` and `can()`, and each writing its events in the same transaction. Long generations stream their output to the screen.

Student-facing

Function	Input	Returns	Events written
`library.browse`	board, class, subject, chapter?	chapters, topics, assets available, the student's mastery per topic	none
`desk.search`	text	matching topics and passages (library + own uploads), best first	none
`desk.openTopic`	topic_id	topic page: summary, level, assets, due recall, open Fix items	none
`desk.learn`	topic_id, mode (understand, story, diagram, use), level?, lang?	the asset (from cache or generated), cache_hit, coins spent	sitting_closed with mode
`desk.ask`	thread_id, text	grounded answer with citations, cache_hit	sitting_closed
`desk.practice.start`	topic_id, kind (mcq, written), count	questions without answers	none
`desk.practice.answer`	question_id, choice or text, seconds, hint_used	correct, misconception correction, next question; written: queued for marking	item_answered (desk), recall_answered

Function	Input	Returns	Events written
`desk.written.result`	attempt_id	key points covered and missed, marks, model answer	remark_given (desk)
`desk.fix.next`	topic_id?	the next open Fix item	none
`desk.recall.due`	none	due recall questions	recall_answered
`desk.plan.make`	goal (test date, subjects)	a week plan	plan_item_set
`desk.plan.tick`	item_id	updated plan	plan_item_done
`uploads.create`	file	upload_id, status	upload_added
`uploads.status`	upload_id	processing state, topics found	none
`desk.flag`	asset_id or question_id, reason	thanks	content_flagged
`session.feedback`	session_id, feel, focus	saved	session_feedback

Staff-facing

Function	Who	Does
`review.queue` / `review.decide`	DHI subject reviewers	approve, edit or retire generated assets and questions
`grading.queue` / `grading.mark`	teachers	confirm or change low-confidence written marks
`library.ingest`	DHI admin	add a book: extract, split, tag, embed
`pregen.run`	DHI admin	generate missing core assets for a chapter, as a batch
`gateway.costs`	DHI admin	cost per student, hit rate per mode, top cache misses

How the desk plugs into the rest of the dashboard

The desk is not a separate app; every other room hands the student to it at the right moment, and it hands results back.

From	Into the desk	Back out
Exams	a "Fix this" button on every wrong answer opens the Fix step for that question's topic	Fix and recall results update the same topic mastery the papers use
Exam lock	while a paper is open, the desk is closed (existing rule)	none
Daily paper builder	uses the desk's question bank and each student's due recall topics for its 40% review share	answers update `learner_topic_state`
Modules	after a film, "practise this" opens Practise on its topic	module check answers count toward mastery
Study room doubts	"take this to the desk" opens Ask with the doubt's topic	none; doubts stay in the room
Library	a chapter page has "learn", "story", "test me" buttons	none
Habits and plans	a desk plan can create a study habit	ticks count in Habits
Session feedback	the feeling and focus card appears when a sitting ends	feeds E09 and E10
Mood and care	distress words typed into the desk go to the care rules	the desk replies gently and offers a counselor; the model does not counsel
Coins	the existing wallet; a cache hit is free	spend appears in the wallet
Growth report	none	metrics A10 to A14, A02 by topic, Fix and recall results

Quality and safety

A wrong explanation reused a thousand times is a thousand wrong lessons, so every shared asset passes checks before it is reused.

Automated checks on every generated asset

- It cites at least one library passage, and its key claims appear in those passages (a second, small model checks this).
- It fits the schema for its kind (a story has 4–6 panels; a diagram spec has labels that exist on the map).
- It passes the copy lint (no ranking words, clinical words or food judgement) and an age-appropriateness check.
- For a question: exactly one correct option, the answer is derivable from the passages, and wrong options are tagged.

An asset that fails is regenerated once, then left for a reviewer. One that passes is served as "new" until reviewed.

Human review. Subject reviewers clear the queue, busiest topics first. Students can flag anything; 3 flags hide it until reviewed.

Evaluation set. For each chapter, 20 fixed questions with expected answers. Every prompt or provider change is run against all of them before it goes live.

Safety

- Exam lock, and hints-not-answers on assigned work.
- Uploads and web results are treated as data, never as instructions.
- Distress goes to care rules; the model does not counsel.
- No personal data in shared assets or in prompts beyond what the step needs. Model providers are contracted not to train on DHI data.

Build order

The desk is built in six steps that slot into the master roadmap's phases 1 to 4. The library and reuse engine come first because every mode depends on them, and they are what make the desk cheap.

Step	What	Fits master phase	Done when
D1 Map and library	knowledge graph for classes 8–12; ingest books; passages with embeddings; library browse	1	every chapter of the trial board has topics and tagged passages
D2 Gateway and asset store	gateway with tiers, budgets and ledger; `assets`, semantic cache, cache-key rule; coin rules	1	a repeated request returns from cache with 0 coins and no model call

Step	What	Fits master phase	Done when
D3 Question bank and practice	`questions` with tags; generation and review; MCQ practice; Fix items; recall schedule	2	a student can practise, fix and recall any topic of 2 pilot subjects
D4 Learning modes	Understand, story, diagram, use, Ask, Plan, all reading the cache first	2–3	each mode works on every topic of the pilot subjects
D5 Mastery and written marking	`learner_topic_state`; level and question picking; written marking with teacher queue	3	levels change with answers; marking agrees with teachers 85%+
D6 Uploads and pre-generation	uploads flow; private cache; batch pre-generation across all subjects	3–4	an upload can be quizzed; core assets exist for every topic of classes 8–12

Making "all subjects, classes 8–12" realistic. The software is the same for every subject; the work is content. Fill the map and the source passages for every chapter in D1, because that is mostly ingestion. Then pre-generate in the order students use it: the trial class's subjects first, then the rest of 8–12 as batches. Until a topic is pre-generated, the desk still works on it live and stores the result, so coverage grows with use.

Open decisions

- [ ] **Book rights:** which books can DHI ingest for the shared library? Textbooks are copyrighted even when free to download, so commercial reuse needs permission or a licence. Get this confirmed before ingesting.
- [ ] **Boards:** CBSE only first, or CBSE plus specific state boards?
- [ ] **Reviewers:** who reviews generated content per subject, and how many hours a week?
- [ ] **Providers:** which providers for each tier, and do they agree not to train on DHI data?
- [ ] **Target price:** what monthly price per student must the model cost fit inside? The ledger will show what hit rate that needs.